Project Update #2

LoveByte

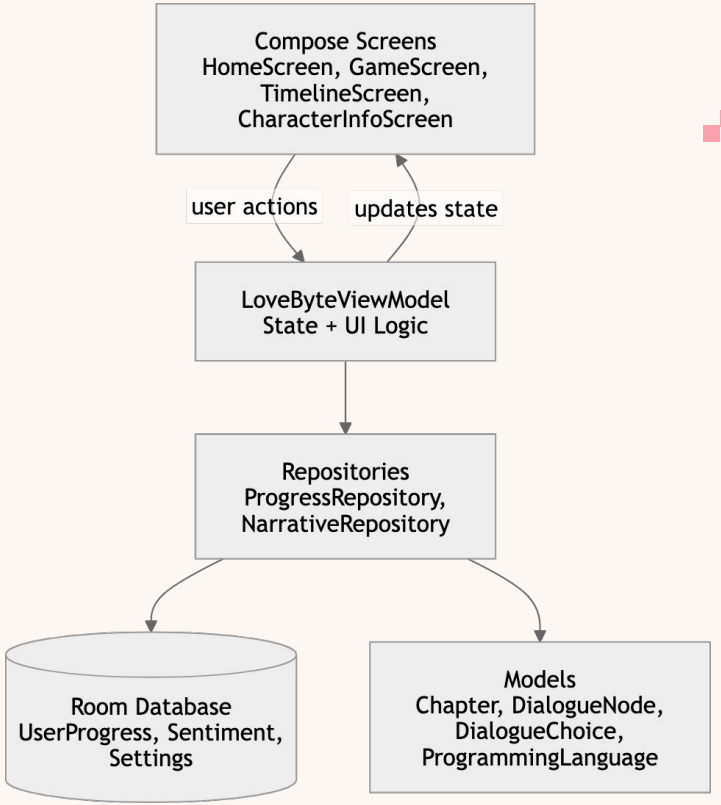
Learn to love — and to program!

01 Architecture

MVVM, flows, dependencies, database schemas, navigation flow)

Architecture: MVVM

LoveByte uses an **MVVM structure**, where the UI is built with **Jetpack Compose screens,** the **LoveByteViewModel** manages app state and user actions, and **repositories** handle data access. The **ViewModel exposes state** to the UI through flows, while **repositories communicate with local data sources such as Room** database tables and static narrative models.



Progression Schemas

The Room database stores **USER_PROGRESS**, which tracks the user's current chapter, dialogue position, and sentiment information for languages so progress can be restored between sessions. Onboarding completion is stored using **SHARED_PREFERENCES**.

SHARED_PREFERENCES	
Boolean	has_seen_onboarding

USER_PROGRESS	
String	language
Int	chapterId
Int	dialogueIndex
Int	lovePoints
Int	friendPoints
Int	hatePoints

Narrative Schemas

The narrative system is structured around modular dialogue and progression units. **Chapter** represents a playable story segment and links to dialogue using a starting node ID. **DialogueNode** stores individual lines of dialogue and supports both linear progression and branching choices. **DialogueChoice** allows users to make decisions that affect the flow of the story and modify sentiment values.

CHAPTER	
Int	id
String	title
Int	startNodeId
Boolean	isLocked
String	difficulty
String	description
ProgrammingLanguage	language
Float	progress

DIALOGUE_NODE	
Int	id
String	speaker
String	text
String	emotion
Int	nextNodeId
List<DialogueChoice>	choices
String	triggerEvent

DIALOGUE_CHOICE	
String	choiceText
Int	targetNodeId
Int	lovePoints
Int	friendPoints
Int	hatePoints

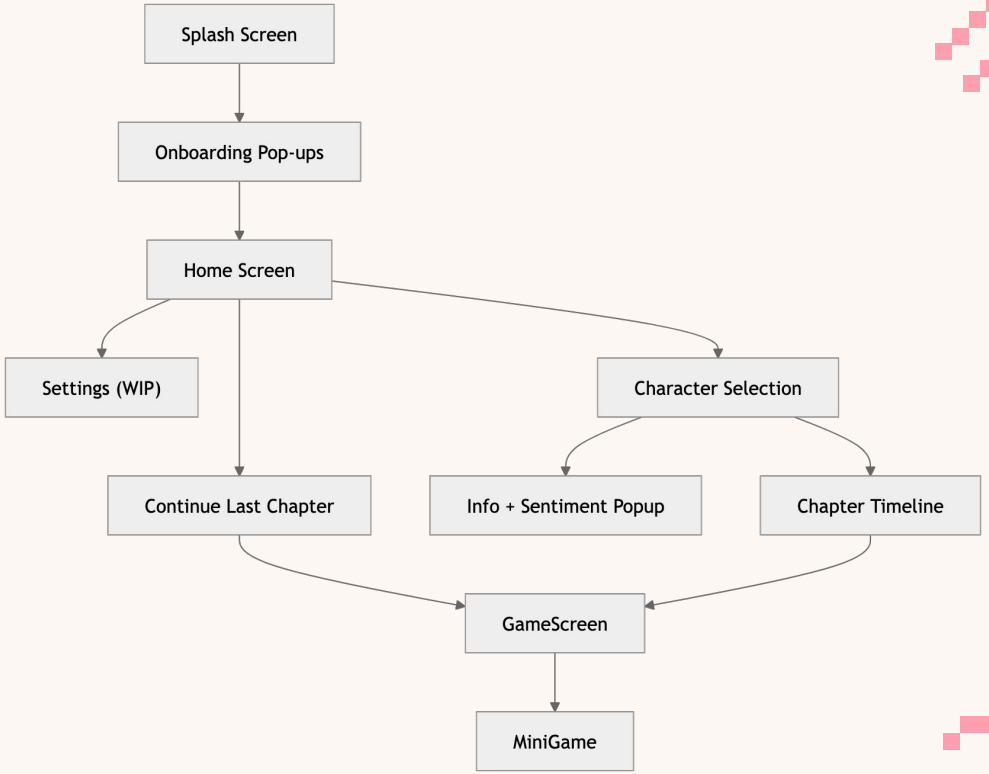
Navigation Flow

> The app begins on the **Splash Screen.**

> New users see onboarding popups before choosing their programming language proficiency, which is on the **Home screen**.

> After selecting a language, users are placed into the correct chapter timeline. From the **Timeline**, they can open a **Chapter**, play through dialogue in the **Game Screen**, complete **Minigames**, and return to the **Timeline**.

> Users can also view character information and current sentiment stats



02 Tests

Debugging & testing strategy

DUAL-LAYER TESTING STRATEGY

LOCAL UNIT TESTING

> Validates app's "brain" in isolation
> Focused on LoveByteViewModel for now
> Created with JUnit 4 and Mockito

Goal: 100% reliability for data transformations and logic without framework interference

INSTRUMENTATION TESTING

> Verifies UI/UX integrity
> Utilizes the Android Compose Test Library

Goal: Verification of state synchronization and navigation across configuration changes.

LOCAL UNIT TESTING

Architectural Refactoring

- Moved data transformations (Weather Mapping & Sentiment) out of the AndroidViewModel lifecycle.
- Bypassed DatabaseProvider and Context dependencies
- No need to wait for an emulator

Key Validation Cases

- Weather Logic: Verifies API string translation (e.g., "Thunderstorm" → "stormy").
- Sentiment Clamping: Ensures sentiment scores stay strictly within the 0–50 range

INSTRUMENTATION TESTING

CharSelectScreen

- **State Sync:** Automated verification that cycling the HorizontalPager (Character Selection) correctly updates the currentLanguage state in the ViewModel
- **Configuration Resilience:** Verified that the dialogueIndex remains consistent during Activity recreation
- **Navigation Integrity:** Tested chapter transitions to ensure no "dead-ends" occur in the narrative flow.

Testing Tech Stack

- **JUnit 4**
  - Validates stateless data logic
- **Mockito Kotlin**
  - Mocking SharedPreferences and Android Application Context
- **Compose UI Test**

03 | Team workflow breakdown

Breakdown

Anna

- CREATED MINI-GAME #2: LIGHT SENSOR MINIGAME
- CREATED “PUBLIC” AND PRIVATE MODES FOR FIRST GAME
- “CUTE-IFIED” THE UI
- MADE UI MORE RESPONSIVE
- BASIC TEST IMPLEMENTATION

Kaylin

- CREATED MINI-GAME #3: EFFICIENCY STEPPER
- CREATED ONBOARDING STEPS
- ADDED SENTIMENT TO DATA MODEL AND UI
- FIXED SOME CHAPTER BUGS

04 Goals

Remaining, stretch goals (progress / descoped)

GOALS {& stretch goals}

01 Sentiment
Make sentiments more visible and obvious

02 Storytelling
Rename chapters to match content, create “endings”

03 Setting Screen
Allow users to revisit permissions

04 Sound
Add sound and music to game

05 Languages
Add more languages and make curriculums

06 Sprites
Create custom sprites to add more personality

AI Use

- **Translating “8-bit” to Android**
- **Preliminary testing**
  - Mockito suggestion after NullPointerError
- **Feature refinement**
- **Last-resort debugging**
- **“Standard practice” guidance**

LoveByte

Questions?